CSE STUDY HUB

C++ Programming

UNIT 3 · Modern C++20 Language Features

What this unit is about

This unit builds understanding of **Modern C++20 Language Features**. These notes are designed for quick learning and exam revision: learn the meaning, understand the working, practise an example, and finish with comparisons and key terms.

Syllabus coverage checklist

1. Type deduction with auto and decltype

2. constexpr and compile-time computation

3. Initialization: uniform/list initialization and narrowing prevention

4. RAII and resource ownership

5. Smart pointers: unique_ptr, shared_ptr and weak_ptr

6. Move semantics, lvalues/rvalues, std::move and move constructors

7. Perfect forwarding and universal references

8. Utility types: optional, variant, any, span and string_view

How to study: Read each topic once for meaning, underline definitions and formulas, reproduce diagrams/steps without looking, then answer the revision questions on Page 5.

Exam answer pattern

For a 5-mark answer: (1) definition, (2) labelled diagram or formula, (3) working/components in steps, (4) example/application, and (5) one advantage, limitation or comparison.

UNIT 3 NOTES · PART 1

Type deduction with auto and decltype

Type deduction with auto and decltype is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

constexpr and compile-time computation

constexpr and compile-time computation is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Initialization: uniform/list initialization and narrowing prevention

Initialization: uniform/list initialization and narrowing prevention is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Modern C++20 Language Features, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

UNIT 3 NOTES · PART 2

RAII and resource ownership

RAII and resource ownership is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Smart pointers: unique_ptr, shared_ptr and weak_ptr

Smart pointers apply RAII to dynamic memory. unique_ptr provides exclusive ownership, shared_ptr shares ownership through a reference count, and weak_ptr observes a shared object without keeping it alive.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Move semantics, lvalues/rvalues, std::move and move constructors

Move semantics transfer resources from a temporary or expiring object instead of copying them. A move constructor/assignment can make vector/string operations cheaper; std::move enables move overload selection but does not move by itself.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Modern C++20 Language Features, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

UNIT 3 NOTES · PART 3

Perfect forwarding and universal references

Perfect forwarding and universal references is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Utility types: optional, variant, any, span and string_view

Utility types: optional, variant, any, span and string_view is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Modern C++20 Language Features, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

QUICK REVISION & EXAM PRACTICE

Key takeaways

• **Type deduction with auto and decltype** — know the definition, core working and one example.

• **constexpr and compile-time computation** — know the definition, core working and one example.

• **Initialization: uniform/list initialization and narrowing prevention** — know the definition, core working and one example.

• **RAII and resource ownership** — know the definition, core working and one example.

• **Smart pointers: unique_ptr, shared_ptr and weak_ptr** — know the definition, core working and one example.

• **Move semantics, lvalues/rvalues, std::move and move constructors** — know the definition, core working and one example.

• **Perfect forwarding and universal references** — know the definition, core working and one example.

• **Utility types: optional, variant, any, span and string_view** — know the definition, core working and one example.

Worked-answer framework

Scenario: A question asks you to explain or apply a concept from Modern C++20 Language Features. **Step 1:** identify the relevant term and define it. **Step 2:** write the formula, components or ordered process. **Step 3:** apply it to the given data/scenario. **Step 4:** state the result and justify it. **Step 5:** mention one limitation or precaution.

Self-test questions

Q1. Define Type deduction with auto and decltype and explain its importance.

Q2. Explain the working/steps involved in constexpr and compile-time computation.

Q3. Differentiate between two related concepts from Modern C++20 Language Features.

Q4. Give one practical application of Utility types: optional, variant, any, span and string_view and discuss one limitation.

Q5. Write a structured 5-mark answer covering the main ideas of Modern C++20 Language Features.

Final checklist: I can define every topic · explain the process without notes · reproduce formulas/diagrams · solve one example · compare related concepts.